

Лекция 5. Функции

1) Актуальность темы

Функции – главный инструмент структурирования кода. Они позволяют разбить программу на логические блоки, избежать дублирования и многократно использовать одни и те же действия. Как только вы научитесь выделять функции, сложность проектов перестанет расти бесконтрольно.

2) Определение и вызов функции

Функция объявляется ключевым словом `def`, за которым идёт имя и круглые скобки:

```
python
def greet():
    print("Привет!")
```

Вызов – просто по имени: `greet()`. Без скобок вызова не произойдёт, вы лишь получите ссылку на объект функции.

3) Параметры и аргументы

Данные передаются через параметры, которые указываются при определении:

```
python
def greet(name):
    print(f"Привет, {name}!")
```

При вызове передаётся аргумент: `greet("Анна")`. Количество аргументов должно совпадать с количеством параметров, если не заданы значения по умолчанию.

Значения по умолчанию позволяют делать параметры необязательными:

```
python
def greet(name="Гость"):
    print(f"Привет, {name}!")
```

```
greet()      # "Привет, Гость!"
greet("Иван") # "Привет, Иван!"
```

Именованные аргументы повышают читаемость:

```
python
def order(product, price, quantity=1):
    total = price * quantity
    print(f'{product}: {total} руб.")
```

```
order(price=100, product="Книга")
```

4) Возврат результата

Функция может возвращать значение с помощью return. Как только встречается return, выполнение функции прекращается:

```
python
```

```
def add(a, b):  
    return a + b
```

```
result = add(3, 5) # result = 8
```

Можно вернуть несколько значений, упаковав их в кортеж:

```
python
```

```
def min_max(numbers):  
    return min(numbers), max(numbers)
```

```
mn, mx = min_max([3, 1, 4]) # mn=1, mx=4
```

5) Области видимости

Переменные, определённые внутри функции, локальны и не видны снаружи. Переменные вне функции – глобальные. Чтобы изменить глобальную переменную внутри функции, требуется ключевое слово `global`, но злоупотреблять им не рекомендуется. Лучше передавать значения через параметры и возвращать результат.

6) Аннотации типов и документация

Хотя типы не обязательны, аннотации делают код понятнее:

```
python
```

```
def multiply(x: int, y: int) -> int:  
    return x * y
```

Строка документации (docstring) сразу после заголовка описывает назначение:

```
python
```

```
def is_even(n):  
    """Возвращает True, если n чётное."""  
    return n % 2 == 0
```

Итоги лекции (что нужно запомнить)

- Функция объявляется через `def`, вызывается по имени со скобками.
- Параметры могут иметь значения по умолчанию.
- `return` завершает функцию и возвращает результат.
- Локальные переменные изолированы внутри функции.
- Для документации используют docstring, для подсказок – аннотации типов.